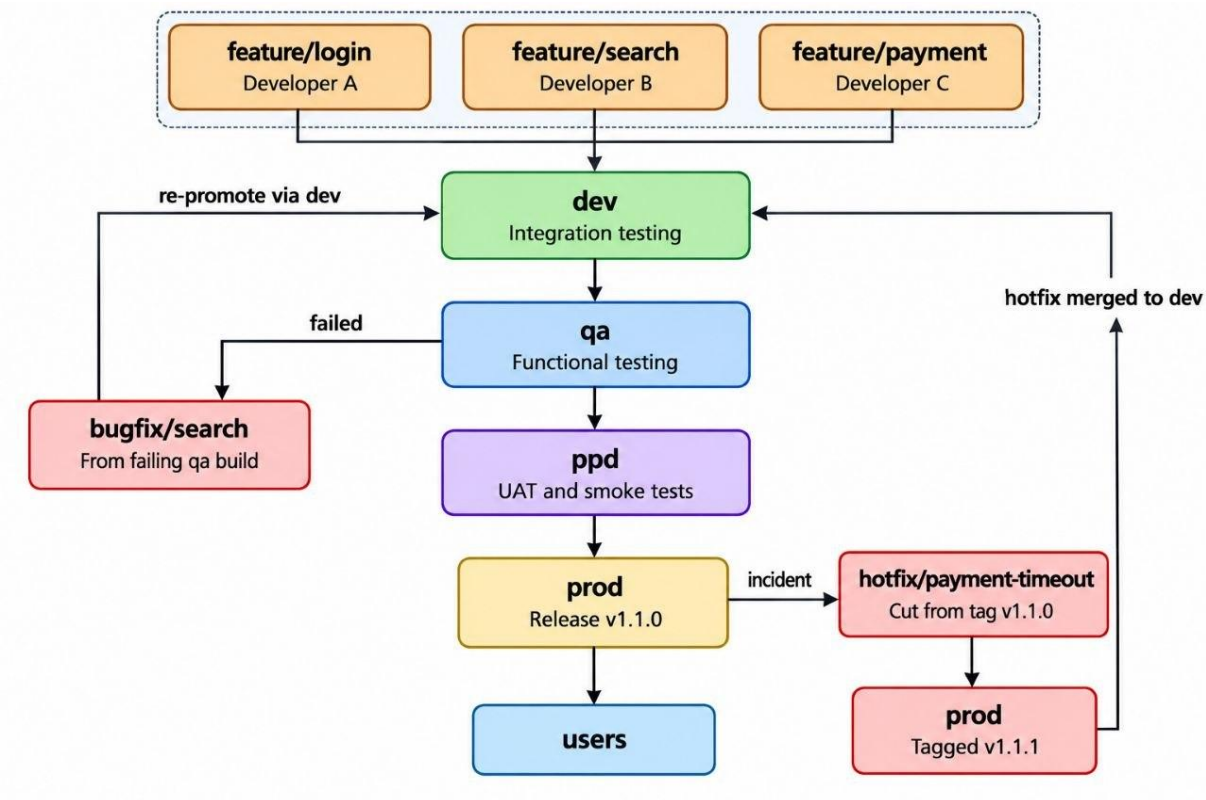
BRANCHING STRATEGY PRACTICAL DEMO



BRANCHING STRATEGY OVERVIEW

1. We create four long-lived branches: dev, qa, ppd, and prod.
2. Multiple developers work on the project simultaneously.
3. Each developer creates a separate feature branch, such as feature/login, feature/search, and feature/payment.
4. While development continues, feature branches can become outdated because other developers are merging code into dev.
5. Developers update their feature branches with the latest changes from dev.
6. Developers raise Pull Requests (PRs) for code review before merging their changes.
7. Once approved, feature branches are merged into dev for integration testing.
8. After successful integration testing, code is promoted from dev to qa.
9. The QA team performs functional testing and identifies bugs.
10. Developers create a bugfix branch to fix the identified issue.
11. After fixing the bug, the code is merged into dev and promoted to qa again.
12. Once QA testing passes, the code is promoted from qa to ppd.
13. The team performs final validation in the pre-production environment.
14. After successful validation, the code is promoted from ppd to prod.

TYPES OF BRANCHES

1. long-lived branch
2. short-lived branch

FOR LONG-LIVED BRANCHES:

1. dev (development and integration)
2. qa (Quality assurance and functional testing)
3. ppd (pre-production validation and UAT)
4. prod (Production-ready code)

FOR SHORT-LIVED BRANCHES:

1. feature/login
2. feature/search
3. feature/payment
4. bugfix/search-validation
5. hotfix/payment-timeout

Short-lived branches are generally deleted after their changes have been merged and are no longer needed.

BRANCHING STRATEGY — DETAILED EXPLANATION

1. First, we create the long-lived branches dev, qa, ppd, and prod. Initially, all four branches contain the same application code.
2. When new functionality is required, developers create feature branches such as feature/login, feature/search, and feature/payment.
3. Assume three developers are working on the project: Developer A, Developer B, and Developer C.
4. Each developer initially works on their local development environment.
5. Once a feature is ready, the developer commits the changes and pushes the code to their respective remote feature branch.
6. Developers raise Pull Requests to merge their feature branches into dev. The team reviews the code and approves the PR before merging.
7. Once the code is merged into dev, the CI/CD pipeline runs. After the required checks pass, the application is deployed to the development environment, represented in this practical by Kubernetes Server 1. The team performs integration testing to verify that the different application components work together correctly.

8. After successful integration testing, a senior team member raises a PR to merge dev into qa.
9. Once the PR is approved and merged, the CI/CD pipeline runs for the qa branch and deploys the application to the QA environment, represented by Kubernetes Server 2.
10. The QA team performs functional testing and other required tests to verify that the application behaves as expected.
11. Suppose the QA team discovers a bug in the search functionality. A developer must create a branch named bugfix/search-validation to fix the issue.
12. In this practical, we create the bugfix branch from dev because dev is our primary integration branch.
13. However, if the fix must be based on the exact release currently being tested in QA, create the bugfix branch from the relevant release or QA commit instead. This avoids unintentionally including unrelated changes that are already present in dev.
14. The developer fixes the issue, tests the changes, commits them, and pushes the bugfix branch to GitHub.
15. A PR is raised to merge bugfix/search-validation into dev. After review and approval, the fix is merged.
16. The updated code is promoted from dev to qa through another PR. The CI/CD pipeline deploys the updated code to the QA environment, where the fix is tested again.
17. Once the QA team confirms that the fix works and all required tests pass, a PR is raised to merge qa into ppd.
18. PPD stands for pre-production. This environment is used for final validation and may also be used for User Acceptance Testing (UAT). It is commonly referred to as a staging environment.
19. The purpose of this environment is to validate the application before production deployment, using production-like configuration and infrastructure wherever possible.
20. When the PR from qa to ppd is approved and merged, the CI/CD pipeline deploys the application to the pre-production environment, represented by Kubernetes Server 3. The team performs UAT, smoke tests, and other required validation.
21. If all required tests pass, a final PR is raised to merge ppd into prod.
22. After approval and merging, the production deployment pipeline deploys the application to the production environment. A release tag is created against the intended production commit.
23. A release identifies a specific version of the application that is ready for deployment. Depending on the delivery process, it may also include packaged artifacts, release notes, and other release information.

24. After successful deployment, users can access the application in the production environment.

Handling production incidents — Hotfix workflow

25. Suppose a critical incident occurs in production, such as payment requests timing out. The team prioritizes investigating and resolving the incident.
25. A developer creates a hotfix branch from the current production branch:
26. hotfix/payment-timeout
27. The branch name has two parts:
    1. hotfix — Indicates an urgent fix for a production issue.
    2. payment-timeout — Describes the specific incident being addressed.
28. The developer investigates the issue, fixes the code, tests the change, commits it, and pushes the hotfix branch. Then two important integrations take place:
29. A PR is raised to merge hotfix/payment-timeout into prod. After review and approval, the fix is merged and deployed through the production pipeline.
30. A PR is also raised to merge the hotfix into dev, ensuring that future development does not lose the production fix. The hotfix may also need to be merged into other maintained release branches, depending on the project.
31. Remember these two key rules:
    1. 1.QA bug: Create a bugfix branch from the appropriate source branch for the fix, integrate the fix, and promote it through testing.
    2. 2.Production incident: Create a hotfix branch from the production code, merge the fix into production through the approved process, and back-merge it into dev.
32. After the production fix is merged and deployed, create a new release tag, such as v1.1.1, pointing to the intended production commit. This identifies the new production version.

PRACTICAL ON BRANCHING STRATEGY

PHASE 1: CREATE THE REPOSITORY AND INITIAL APPLICATION

1.CREATE A REPOSITORY ON THE GITHUB

1. ecommerce-app

2.CLONNING THE REPO INTO THE VS-CODE

1. git clone <YOUR_GITHUB_REPOSITORY_URL>
2. cd ecommerce-app

3.CREATE THE APPLICATION

1. mkdir src
2. cd src
3. vim index.html
   <h1> DevOps Ecommerce Application </h1>
   <h2> Production Version 1.0 </h2>
4. cd ../
5. git add .
6. git commit -m "Add Initial Ecommerce app"
7. git push origin main

4.CHECK IT ON THE GITHUB

PHASE-2 RENAMING THE MAIN BRANCH AS A PROD BRANCH

1.RENAME THE LOCAL BRANCH

1. git branch -m main prod

2.PUSH THE NEW BRANCH

1. git push -u origin prod

3.CHANGE THE DEFAULT BRANCH ON GITHUB

1. go to repo settings
2. click on Default branch
3. change main to prod
4. then update

4.DELETE THE OLD REMOTE BRANCH

1. git push origin --delete main

PHASE 3: CREATE THE FIRST RELEASE

1. git switch prod
2. git pull origin prod
3. git tag -a v1.0.0 -m "Initial Production release v1.0.0"
4. git push origin v1.0.0
5. git log --oneline

The tag v1.0.0 identifies the initial production commit.

PHASE 4: Create the Long-Lived Branches

CREATE THE PPD BRANCH

1. git switch prod
2. git switch -c ppd
3. git push -u origin ppd

CREATE THE DEV BRANCH

1. git switch prod
2. git switch -c dev
3. git push -u origin dev

CREATE THE QA BRANCH

1. git switch prod
2. git switch -c qa
3. git push -u origin qa

Note:
All these branches having the same code V1.0.0

PHASE 5: DEVELOPER A — LOGIN FEATURE

CREATE THE FEATURE BRANCH

1. git switch dev
2. git pull origin dev
3. git switch -c feature/login

CREATE THE LOGIN FEATURE FILE

1. touch login.txt
2. git add .
3. git commit -m "Add login Feature"
4. git push -u origin feature/login

CREATING PULL REQUEST TO MERGE CODE { FEATURE/LOGIN - DEV }

1. click on compare and pull request
2. base - dev
3. compare - feature/login
4. create the pull request

COMPLETE THE MERGE REQUEST INTO DEV OF FEATURE/LOGIN

1. click on merge pull request
2. Confirm merge

DELETE THE FEATURE/LOGIN BRANCH

1. delete the feature/login branch

OR

1. git switch dev
2. git pull origin dev
3. git branch -d feature/login
4. git push origin --delete feature/login

The feature branch is no longer needed after its changes have been merged.

PHASE 6: DEVELOPER B — SEARCH FEATURE

CREATE THE FEATURE BRANCH

1. git switch dev
2. git pull origin dev
3. git switch -c feature/search

CREATE THE CORRECT SEARCH FILE

1. touch search.txt
2. git add .
3. git commit -m "Add search feature"
4. git push -u origin feature/search

UPDATE THE FEATURE BRANCH IF DEV CHANGES

1. git fetch origin
2. git switch feature/search
3. git merge origin/dev
4. git push origin feature/search

{If other developers have merged changes into dev while search development is in progress}

CREATING PULL REQUEST TO MERGE CODE { feature/search - Dev }

1. click on new pull request

2. base - dev
3. compare - feature/search
4. create the pull request

COMPLETE THE MERGE REQUEST INTO DEV OF FEATURE/SEARCH

1. click on merge pull request
2. Confirm merge

DELETE THE FEATURE/SEARCH BRANCH

1. delete the feature/search branch

OR

1. git switch dev
2. git pull origin dev
3. git branch -d feature/search
4. git push origin --delete feature/search

PHASE 7: DEVELOPER C — PAYMENT FEATURE

CREATE THE FEATURE BRANCH

1. git switch dev
2. git pull origin dev
3. git switch -c feature/payment

CREATE THE PAYMENT FEATURE FILE

1. touch payment.txt
2. git add .
3. git commit -m "Add payment Feature"
4. git push -u origin feature/payment

CREATING PULL REQUEST TO MERGE CODE { feature/payment - Dev }

1. click on new pr
2. base - dev
3. compare - feature/payment
4. create the pull request

COMPLETE THE MERGE REQUEST INTO DEV OF FEATURE/PAYMENT

1. click on merge pull request
2. Confirm merge

DELETE THE FEATURE/PAYMENT BRANCH

1. delete the feature/payment branch
   OR
2. git switch dev
3. git pull origin dev
4. git branch -d feature/payment
5. git push origin --delete feature/payment

PHASE 8: DEV ENVIRONMENT DEPLOYMENT

At this stage, dev contains:

1. Original application
2. Login feature
3. Search feature
4. Payment feature

When a feature PR is merged into dev, the CI pipeline should validate the changes. After the merge, the configured deployment pipeline can deploy the application to Kubernetes Server 1.

The team performs integration testing in the DEV environment.

PHASE 9: PROMOTE DEV TO QA

CREATING PULL REQUEST TO MERGE CODE (DEV → QA)

1. Click New pull request.
2. Base: qa
3. Compare: dev
4. Create the pull request.

COMPLETE THE MERGE REQUEST INTO QA OF DEV BRANCH

1. click on merge pull request
2. Confirm merge

IN QA BRANCH

1. After the code is merged into qa, the configured CI/CD pipeline runs.
2. The application is deployed to Kubernetes Server 2.
3. The QA team performs functional testing and other required tests.

UPDATE THE QA BRANCH

1. git switch qa
2. git pull origin qa

AT THESE POINT OUR CODE IS...

1.DEV

1. original application
2. Login
3. Search
4. Payment

2.QA

1. original application
2. Login
3. Search
4. Payment

3.PPD

1. Original Application

4.PROD

2. Original Application

PHASE 10: QA FINDS A BUG

Suppose the QA team identifies a bug in the search feature.

For this demonstration, we will create the bugfix branch from dev. This assumes the fix is compatible with the code being tested in QA. If QA is testing an older release, create the fix from that release instead and merge it back into dev.

WORKING ON THE BUGFIX/SEARCH-VALIDATION BRANCH

Update the DEV branch.

1. git switch dev
2. git pull origin dev

Create the bugfix branch.

1. git switch -c bugfix/search-validation

Create a demonstration file

2. touch bugfix.txt
3. git add .
4. git commit -m "Fix empty search validation"
5. git push -u origin bugfix/search-validation

Note:
In a real application, fix the actual search validation code rather than creating only a placeholder file.

CREATING PULL REQUEST TO MERGE CODE { bugfix/search-validation - Dev }

1. click on new pull request
2. base - dev
3. compare - bugfix/search-validation
4. create the pull request

COMPLETE THE MERGE REQUEST INTO DEV OF BUGFIX/SEARCH-VALIDATION

1. click on merge pull request
2. Confirm merge

DELETE THE FEATURE/SEARCH BRANCH

1. delete the bugfix/search-validation branch

OR

1. git switch dev
2. git pull origin dev
3. git branch -d bugfix/search-validation
4. git push origin --delete bugfix/search-validation

PROMOTE THE FIXED CODE (DEV → QA)

1. Open a new pull request.
2. Base: qa
3. Compare: dev
4. Create the PR.

COMPLETE THE MERGE REQUEST INTO DEV OF QA

1. click on merge pull request
2. Confirm merge

The configured CI/CD pipeline deploys the updated code to Kubernetes Server 2.
The QA team retests the search feature and performs regression testing.

PHASE 11: PROMOTE QA TO PPD (PRE-PRODUCTION ENVIRONMENT)

After the QA team confirms that the application passes the required tests, promote the code to pre-production.

CREATING PULL REQUEST TO MERGE CODE { QA - PPD }

1. Click on pull request
2. click on new pull request
3. base - ppd
4. compare - qa
5. Title - promoting code from QA to PPD
6. create a pull request

COMPLETE THE MERGE REQUEST INTO PPD OF QA

1. click on merge pull request
2. Confirm merge

UPDATE THE PPD BRANCH

1. git switch ppd
2. git pull origin ppd

The configured CI/CD pipeline deploys the application to Kubernetes Server 3.

PPD VALIDATION

Perform the required checks:

1. User Acceptance Testing (UAT), where applicable.
2. Smoke testing.
3. Final application validation.
4. Production-readiness checks.

PPD is the pre-production environment, commonly referred to as staging. It is used to validate the application before production deployment.

AT THESE POINT OUR CODE IS...

1.DEV

1. original application
2. Login
3. Search

4. Payment

2.QA

1. original application
2. Login
3. Search
4. Search Fixed (bugfix.txt)
5. Payment

3.PPD

1. original application
2. Login
3. Search
4. bugfix.txt
5. Payment

4.PROD

1. Original Application

PHASE 12: CONFIGURE PROD BRANCH PROTECTION

CREATE A BRANCH RULESET ON GITHUB

1. Open repository Settings.
2. Navigate to Rules → Rulesets.
3. Click New ruleset.
4. Select New branch ruleset
5. Ruleset name: PROD-PR
6. Enforcement status: Active
7. Target the prod branch.
8. Branch rules
    1. Restrict deletions.
    2. Block force pushes.
    3. Require a pull request before merging.
    4. Require approvals, if appropriate.
    5. Require successful CI checks, if configured.
9. save the ruleset

Note:
Do not leave enforcement disabled if you want the rules to protect the production branch.

PHASE 13: PROMOTE PPD TO PROD

CREATING PULL REQUEST TO MERGE CODE { PPD - PROD }

1. Click on pull request
2. click on new pull request
3. base - prod
4. compare - ppd
5. Title - PPD-PROD
6. create a pull request

COMPLETE THE MERGE REQUEST INTO PROD OF PPD

1. click on merge pull request
2. Confirm merge

UPDATE THE PROD BRANCH IN VS CODE

1. git switch prod
2. git pull origin prod

CREATE THE NEW RELEASE

1. git log --oneline
2. git tag -a v1.1.0 -m "Production release v1.1.0"
3. git push origin v1.1.0
4. git log --oneline
   tag: v1.0.0
   tag: v1.1.0

The tag v1.1.0 identifies the intended production release commit. Confirm the production deployment has succeeded before announcing the release as deployed.

PHASE 14: CRITICAL PRODUCTION INCIDENT — HOTFIX

CREATE THE HOTFIX BRANCH FROM PROD

1. git switch prod
2. git pull origin prod
3. git switch -c hotfix/payment-timeout

FIX THE PRODUCTION ISSUE

1. touch hotfix-payment.txt
2. git add .

3. git commit -m "Fix production payment timeout"
4. git push -u origin hotfix/payment-timeout

CREATING PULL REQUEST (HOTFIX → PROD)

1. Click on pull request
2. click on new pull request
3. base - prod
4. compare - hotfix/payment-timeout
5. Title - Fix production payment timeout
6. create a pull request

COMPLETE THE MERGE REQUEST INTO PROD OF HOTFIX

1. click on merge pull request
2. Confirm merge

UPDATE PROD AND CREATE THE HOTFIX RELEASE

1. git switch prod
2. git pull origin prod
3. git log --oneline
4. git tag -a v1.1.1 -m "Payment timeout hotfix"
5. git push origin v1.1.1
6. git log --oneline
   tag: v1.0.0
   tag: v1.1.0
   tag: v1.1.1

MERGE THE HOTFIX BACK INTO DEV

1. Click on pull request
2. click on new pull request
3. base - dev
4. compare - hotfix/payment-timeout
5. Title - Back-merge payment timeout hotfix into DEV
6. create a pull request

COMPLETE THE MERGE REQUEST INTO DEV OF HOTFIX

1. click on merge pull request
2. Confirm merge

UPDATE THE LOCAL DEV BRANCH

1. git switch dev

2. git pull origin dev

DELETE THE HOTFIX BRANCH

1. git push origin --delete hotfix/payment-timeout
2. git branch -d hotfix/payment-timeout

RELEASE VERSIONS

1. v1.0.0 — Initial release.
2. v1.1.0 — Feature release.
3. v1.1.1 — Payment-timeout hotfix release.

PHASE 15: Understand CI and CD Triggers

Pull Request Created or Updated

1. CI runs tests, builds, and other configured checks.
2. The PR can be blocked from merging until required checks pass.
3. Production deployment is not normally triggered by every PR.

Pull Request Merged

1. The pipeline for the target branch can run.
2. The application can be deployed to the corresponding environment if deployment is configured.
3. Production deployments should follow the required approval and release process.

Important:
GitHub branches and PRs do not automatically create CI/CD pipelines. You must configure GitHub Actions, Jenkins, or another CI/CD tool.

AT THESE POINT OUR APPLICATION CODE IS...

1.DEV

1. original application
2. Login
3. Search
4. Payment

2.QA

1. original application
2. Login
3. Search
4. Search Fixed (bugfix.txt)
5. Payment

3.PPD

1. original application
2. Login
3. Search
4. bugfix.txt
5. Payment

4.PROD

1. original application
2. Login
3. Search
4. bugfix.txt
5. Payment
